

Tecnologico de Costa Rica
Escuela de Computacion
Principios de Sistemas Operativos

Tercer Proyecto

Simulador de AWS S3 sobre sockets

Profesor: Armando Arce Orozco

Estudiantes:

Nicole Parra, 2023223291

Alfredo Mercado, 2023377984

Junio 2026

Índice

1. Introduccion	1
2. Descripcion del problema	1
3. Comandos implementados	1
4. Definicion de estructuras de datos utilizadas	2
4.1. URI S3	2
4.2. Entrada de objeto	2
4.3. Hueco libre	3
4.4. Directorio del bucket	3
5. Descripcion detallada de componentes	3
5.1. src/common.c	3
5.2. src/bucket.c	4
5.3. src/server.c	4
5.4. src/client.c	4
6. Mecanismo de creacion de archivos	4
7. Mecanismo de comunicacion con el servidor	5
7.1. Carga de un archivo	5
7.2. Descarga de un archivo	5
7.3. Listados y sincronizacion	5
8. Pruebas de funcionamiento	6
9. Pruebas de rendimiento	6
10. Analisis de resultados	8
11. Limitaciones	8
12. Compilacion y ejecucion	9
13. Conclusiones	9

1. Introduccion

El proyecto implementa una simulacion de almacenamiento remoto de objetos al estilo AWS S3. La solucion contiene dos ejecutables principales: `aws-s3_server`, que administra los buckets en el servidor, y `aws-s3`, que funciona como cliente de linea de comandos. Ambos programas estan escritos en C y se comunican por sockets TCP.

La idea central es representar cada bucket como un unico archivo local dentro del servidor. En ese archivo se guarda primero un bloque de directorio con metadatos y, despues de ese bloque, el contenido binario de los objetos. Las carpetas no existen como estructuras fisicas; se simulan mediante prefijos en las claves, igual que en S3. Por ejemplo, `fotos/2026/playa.jpg` es una sola clave con barras diagonales, no una jerarquia real de directorios dentro del bucket.

2. Descripcion del problema

El objetivo es construir un sistema cliente-servidor que permita gestionar archivos locales y remotos usando comandos similares a `aws s3`. El cliente debe recibir los argumentos desde terminal, interpretarlos como rutas locales o rutas `s3://bucket/clave`, y enviar la operacion correspondiente al servidor.

El servidor debe mantener multiples buckets. Cada bucket se guarda en un unico archivo, con un directorio interno que registra las claves de los objetos, el offset donde inicia cada contenido y el tamaño en bytes. Este requisito evita guardar cada objeto como un archivo separado y obliga a administrar espacio libre dentro del archivo del bucket.

Cuando un objeto se reemplaza, hay dos casos:

- Si el nuevo archivo tiene el mismo tamaño, se sobrescribe en el mismo offset.
- Si el tamaño cambia, el espacio anterior se marca como libre y el nuevo contenido se escribe en un hueco disponible o al final del bucket.

Ademas, al eliminar objetos se debe actualizar la tabla de objetos y agregar el espacio que ocupaban a una lista de huecos reutilizables. La solucion implementada usa primer ajuste para asignar espacio libre: se toma el primer hueco suficientemente grande y, si sobra espacio, el hueco se reduce.

3. Comandos implementados

Comando	Descripcion
<code>ls</code>	Lista todos los buckets cuando no recibe argumentos, o lista objetos y prefijos dentro de un bucket.
<code>mb</code>	Crea un bucket nuevo usando una ruta <code>s3://bucket</code> .

Comando	Descripcion
cp	Copia archivos local a S3, S3 a local o S3 a S3. Soporta -recursive para directorios o prefijos.
mv	Mueve un archivo u objeto mediante copia seguida de eliminacion del origen. Tambien soporta -recursive .
rm	Elimina un objeto. Con -recursive elimina todos los objetos que empiezan con el prefijo indicado.
sync	Sincroniza local a S3 o S3 a local. Con -delete elimina en el destino lo que ya no existe en el origen.
rb	Elimina un bucket vacio. Con -force permite eliminarlo aunque tenga objetos.

4. Definicion de estructuras de datos utilizadas

Las estructuras principales estan definidas en `include/common.h` y `include/bucket.h`. Se eligieron estructuras simples, con tamaños maximos fijos, porque el enfoque es la administracion de buckets y no una base de datos dinamica de metadatos.

4.1. URI S3

```
typedef struct {
    char bucket[S3_MAX_BUCKET];
    char key[S3_MAX_KEY];
    int is_s3;
} s3_uri_t;
```

`s3_uri_t` guarda el resultado de interpretar una ruta `s3://bucket/clave`. El campo `bucket` contiene el nombre del bucket, `key` contiene la clave o prefijo dentro del bucket, e `is_s3` indica si el argumento era una ruta remota.

4.2. Entrada de objeto

```
typedef struct {
    int used;
    char key[S3_MAX_KEY];
    uint64_t offset;
    uint64_t size;
} bucket_entry_t;
```

Cada `bucket_entry_t` representa un objeto almacenado. La clave es plana y puede contener barras para simular carpetas. `offset` indica el byte dentro del archivo `.bucket` donde empieza el contenido, y `size` indica cuantos bytes deben leerse o transmitirse.

4.3. Hueco libre

```
typedef struct {
    int used;
    uint64_t offset;
    uint64_t size;
} bucket_hole_t;
```

La lista de `bucket_hole_t` permite reutilizar espacio interno cuando se borra un objeto o cuando se reemplaza por otro con diferente tamaño. El algoritmo implementado es primer ajuste.

4.4. Directorio del bucket

```
typedef struct {
    char magic[8];
    uint32_t entry_count;
    uint32_t hole_count;
    bucket_entry_t entries[BUCKET_MAX_OBJECTS];
    bucket_hole_t holes[BUCKET_MAX_HOLES];
} bucket_header_t;
```

El `bucket_header_t` esta al inicio de cada archivo de bucket. Incluye una firma `S3BKT01`, el conteo de objetos activos, el conteo de huecos libres, la tabla de objetos y la tabla de huecos. En esta version se soportan hasta 512 objetos y 512 huecos por bucket.

5. Descripcion detallada de componentes

5.1. `src/common.c`

Este modulo contiene funciones compartidas por cliente y servidor:

- Apertura de sockets cliente y servidor.
- Lectura y escritura exacta de bytes con `read_exact` y `write_exact`.
- Lectura de lineas de control con `read_line`.
- Envio de lineas formateadas con `send_linef`.
- Copia de streams binarios entre archivos y sockets.
- Utilidades de rutas, creacion de directorios y calculo de tamaños.

La separacion es importante porque evita duplicar codigo sensible en cliente y servidor. En especial, las funciones de lectura y escritura exacta protegen contra lecturas parciales del socket.

5.2. `src/bucket.c`

Este modulo implementa la capa de almacenamiento. Sus responsabilidades son:

- Crear y eliminar archivos `.bucket`.
- Leer y escribir el bloque de directorio.
- Insertar objetos desde archivos locales o desde streams recibidos por socket.
- Enviar objetos al cliente.
- Copiar objetos entre buckets.
- Eliminar objetos individuales o prefijos completos.
- Listar buckets y claves.
- Consultar el tamaño de una clave para `sync`.

El bloque de directorio se reescribe despues de cada operacion que modifica metadatos. El contenido de los objetos se conserva en la zona de datos. Si se libera espacio, se agrega un hueco a la tabla para reutilizacion futura.

5.3. `src/server.c`

El servidor crea un socket TCP, escucha en el puerto configurado y acepta conexiones de clientes. Por cada conexion hace `fork`, de modo que cada comando se atiende en un proceso hijo. La conexion procesa una linea de comando, ejecuta la operacion y responde con `OK` o `ERR`.

El protocolo interno usa nombres de comandos mas directos que la interfaz del cliente. Los comandos internos son `CREATE_BUCKET`, `REMOVE_BUCKET`, `LIST`, `LIST_RAW`, `PUT`, `GET`, `COPY`, `DELETE` y `STAT`.

5.4. `src/client.c`

El cliente implementa la interfaz. Primero procesa opciones globales como `-host` y `-port`; luego identifica el comando y separa rutas locales de rutas remotas con `parse_s3_uri`.

Para operaciones recursivas, el cliente recorre directorios locales con `opendir` y `readdir`. Para operaciones sobre prefijos remotos, pide al servidor una lista cruda con `LIST_RAW` y luego copia, descarga o elimina cada clave coincidente.

6. Mecanismo de creacion de archivos

Los buckets se guardan en el directorio `buckets/`. Cada bucket se almacena como un archivo con extension `.bucket`. La creacion sigue estos pasos:

1. El servidor valida el nombre del bucket.
2. Crea el directorio `buckets/` si no existe.
3. Crea el archivo `.bucket` en modo binario.
4. Escribe un `bucket_header_t` inicial con la firma `S3BKT01`, sin objetos y sin huecos.

La zona de datos empieza despues del `bucket_header_t`. Cuando se guarda un objeto nuevo, el servidor busca primero un hueco libre suficiente. Si lo encuentra, escribe ahi; si no, escribe al final del archivo.

7. Mecanismo de comunicacion con el servidor

El protocolo combina lineas de control en texto con payloads binarios. Esto mantiene los comandos legibles y evita corromper archivos que no son texto.

7.1. Carga de un archivo

Para subir un objeto, el cliente envia:

```
PUT bucket key size\n<size bytes exactos del archivo>
```

El servidor lee la linea, reserva espacio en el bucket y luego lee exactamente `size` bytes desde el socket. Al terminar, responde `OK` o `ERR`.

7.2. Descarga de un archivo

Para descargar un objeto, el cliente envia:

```
GET bucket key\n
```

Si el objeto existe, el servidor responde:

```
OK size\n<size bytes exactos del objeto>
```

El cliente crea los directorios locales necesarios y escribe los bytes recibidos en el archivo destino.

7.3. Listados y sincronizacion

`LIST` devuelve una vista amigable con objetos y prefijos. `LIST_RAW` devuelve claves completas y tamaños, separadas por tabulador. Esta segunda variante se usa para `cp -recursive`, `mv -recursive`, `rm -recursive` y `sync`.

8. Pruebas de funcionamiento

El proyecto incluye una prueba automatizada en `tests/smoke.sh`. Esta prueba compila el proyecto, levanta un servidor local, ejecuta los comandos principales y usa `cmp` para validar que los archivos descargados son byte a byte iguales a los originales.

```
make test
```

La secuencia cubierta por la prueba es:

```
./aws-s3 --port 19090 mb s3://demo
./aws-s3 --port 19090 cp tmp-smoke/local/a.txt s3://demo/
./aws-s3 --port 19090 cp tmp-smoke/local s3://demo/copia/ --recursive
./aws-s3 --port 19090 ls s3://demo/
./aws-s3 --port 19090 cp s3://demo/a.txt tmp-smoke/out/a.txt
cmp tmp-smoke/local/a.txt tmp-smoke/out/a.txt
./aws-s3 --port 19090 cp s3://demo/copia/ tmp-smoke/out/copia --recursive
cmp tmp-smoke/local/dir/b.txt tmp-smoke/out/copia/dir/b.txt
./aws-s3 --port 19090 cp s3://demo/copia/ s3://demo/copia2/ --recursive
./aws-s3 --port 19090 mv s3://demo/a.txt s3://demo/movido.txt
./aws-s3 --port 19090 mv s3://demo/copia2/ s3://demo/copia3/ --recursive
./aws-s3 --port 19090 cp s3://demo/copia3/dir/b.txt tmp-smoke/out/b2.txt
cmp tmp-smoke/local/dir/b.txt tmp-smoke/out/b2.txt
./aws-s3 --port 19090 rm s3://demo/copia/ --recursive
./aws-s3 --port 19090 sync tmp-smoke/local s3://demo/sync/ --delete
./aws-s3 --port 19090 sync s3://demo/sync/ tmp-smoke/out/sync
cmp tmp-smoke/local/dir/b.txt tmp-smoke/out/sync/dir/b.txt
./aws-s3 --port 19090 rb s3://demo --force
```

La salida esperada al final de la prueba es:

```
Pruebas smoke completadas
```

9. Pruebas de rendimiento

Para revisar el comportamiento con archivos de diferentes tamaños, se ejecuto una prueba manual de rendimiento en ambiente Unix. La prueba mide carga, descarga e integridad usando archivos binarios de 10 MiB, 50 MiB y 100 MiB, con tres repeticiones por tamaño. La medicion se hizo contra el servidor local por *loopback*; por eso los resultados representan el costo de protocolo, copias de bytes y acceso a disco local, no el rendimiento de una red real.

```
make
rm -rf buckets tmp-perf
mkdir -p tmp-perf/in tmp-perf/out
dd if=/dev/urandom of=tmp-perf/in/10m.bin bs=1M count=10 status=none
dd if=/dev/urandom of=tmp-perf/in/50m.bin bs=1M count=50 status=none
dd if=/dev/urandom of=tmp-perf/in/100m.bin bs=1M count=100 status=none
```

```
./aws-s3_server 19191 > tmp-perf/server.log 2>&1 &
SERVER_PID=$!
sleep 1
./aws-s3 --port 19191 mb s3://perf

echo "archivo,bytes,rep,subida_s,descarga_s,integridad"
# Para cada archivo y repeticion se midio con date +%s%%N.
# La integridad se valido con cmp -s.
./aws-s3 --port 19191 rb s3://perf --force
kill "$SERVER_PID"
```

Los resultados crudos observados fueron:

Archivo	Bytes	Rep.	Subida (s)	Descarga (s)	Integridad
10m.bin	10485760	1	0.0120	0.0109	OK
10m.bin	10485760	2	0.0087	0.0108	OK
10m.bin	10485760	3	0.0091	0.0115	OK
50m.bin	52428800	1	0.0490	0.0462	OK
50m.bin	52428800	2	0.0467	0.0448	OK
50m.bin	52428800	3	0.0341	0.0439	OK
100m.bin	104857600	1	0.0955	0.0927	OK
100m.bin	104857600	2	0.0585	0.0913	OK
100m.bin	104857600	3	0.0670	0.0898	OK

Tabla 2: Resultados crudos de carga, descarga y comparacion con `cmp`.

Con esos datos se calcularon los promedios y el rendimiento efectivo:

Archivo	Tamaño (MiB)	Subida prom. (s)	Descarga prom. (s)	Integridad
10m.bin	10	0.0099	0.0111	3/3 OK
50m.bin	50	0.0433	0.0450	3/3 OK
100m.bin	100	0.0737	0.0913	3/3 OK

Tabla 3: Promedio de tres repeticiones por tamaño.

Archivo	Subida (MiB/s)	Descarga (MiB/s)
10m.bin	1006.7	903.6
50m.bin	1155.6	1111.9
100m.bin	1357.5	1095.7

Tabla 4: Rendimiento efectivo sobre loopback local.

Estas pruebas son importantes porque validan dos aspectos:

- El protocolo transmite bytes exactos y no depende de que el archivo sea texto.
- El bloque de directorio sigue siendo correcto despues de varias inserciones, descargas y eliminaciones.

10. Analisis de resultados

La prueba automatizada, valida que un bucket pueda crearse, recibir archivos, listar contenido, copiar prefijos completos, mover objetos, eliminar prefijos, sincronizar directorios y eliminarse con `-force`.

Las comparaciones con `cmp` son la verificacion mas importante porque comprueban que la descarga produce exactamente los mismos bytes que el archivo original. Esto demuestra que la separacion entre linea de control y payload binario funciona correctamente.

La prueba de rendimiento con archivos de 10 MiB, 50 MiB y 100 MiB mostro tiempos crecientes conforme aumenta el tamaño del archivo. En todas las repeticiones, `cmp` confirmo que el archivo recuperado era identico al original. El mayor rendimiento observado en archivos grandes es esperable porque el costo fijo de abrir conexiones, procesar comandos y actualizar el bloque de directorio se reparte entre mas bytes transferidos.

La sincronizacion compara existencia y tamaño de archivos. Esta decision mantiene el diseño simple y suficiente para el alcance del proyecto.

11. Limitaciones

- Cada bucket soporta hasta 512 objetos y 512 huecos libres.
- Las claves tienen un maximo de 255 caracteres.
- El protocolo separa argumentos por espacios, por lo que no esta pensado para claves remotas con espacios.
- `sync` detecta cambios por tamaño; no compara hashes ni tiempos de modificacion.
- El servidor atiende procesos hijos con `fork`, pero no implementa bloqueo de escritura entre operaciones concurrentes sobre el mismo bucket.

Estas limitaciones no impiden cumplir los comandos solicitados, pero delimitan el alcance real de la implementacion.

12. Compilacion y ejecucion

El proyecto esta preparado para ambiente Unix. La compilacion se realiza con:

```
make
```

Luego se levanta el servidor:

```
./aws-s3_server 9090
```

Y en otra terminal se ejecutan comandos del cliente:

```
./aws-s3 --port 9090 mb s3://demo  
./aws-s3 --port 9090 cp archivo.txt s3://demo/  
./aws-s3 --port 9090 ls s3://demo/  
./aws-s3 --port 9090 cp s3://demo/archivo.txt ./  
./aws-s3 --port 9090 rm s3://demo/archivo.txt  
./aws-s3 --port 9090 rb s3://demo --force
```

Para ejecutar la validacion automatizada:

```
make test
```

13. Conclusiones

La implementacion cumple el objetivo de simular un servicio de almacenamiento de objetos con buckets, claves planas y prefijos. El cliente traduce comandos similares a AWS S3 en mensajes simples sobre sockets, mientras que el servidor administra los metadatos y el contenido dentro de archivos `.bucket`.

El uso de un bloque de directorio fijo simplifica la recuperacion de metadatos y permite mantener cada bucket en un solo archivo. La lista de huecos libres resuelve el problema de reutilizar espacio cuando se eliminan o reemplazan objetos, y el algoritmo de primer ajuste es suficiente para esta escala.

Las pruebas propuestas cubren los comandos principales y validan la integridad de los archivos recuperados. Para evitar problemas de ejecucion en la entrega, el proyecto debe compilarse y probarse en un ambiente Unix con `gcc`, `make` y utilidades POSIX disponibles.